

Maximum Cut Benchmark Evaluation

CSE 318
Artificial Intelligence

Offline Assignment

Student ID: 2205117

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

July 27, 2026

1. Introduction

The Maximum Cut (MAX-CUT) problem is one of the classical NP-hard optimization problems in graph theory. Given an undirected weighted graph, the objective is to divide the vertices into two disjoint partitions such that the sum of the weights of all edges crossing the two partitions is maximized.

Because the problem is NP-hard, computing an optimal solution for large benchmark instances is computationally expensive. Consequently, heuristic and metaheuristic algorithms are widely used to obtain near-optimal solutions within a reasonable running time.

In this assignment, five algorithms were implemented and evaluated on the standard MAX-CUT benchmark graphs:

- Randomized Algorithm
- Greedy Algorithm
- Semi-Greedy Algorithm
- Local Search
- GRASP (Greedy Randomized Adaptive Search Procedure)

The objective is to compare their solution quality across multiple benchmark graphs and evaluate the effectiveness of increasingly sophisticated heuristic techniques.

2. Algorithms

2.1 Randomized Algorithm

The Randomized algorithm constructs a solution by assigning each vertex independently to one of the two partitions with equal probability. No information about the graph structure or edge weights is used during construction.

The algorithm is repeated several times, and the average cut value is reported. Because every partition is generated completely at random, the algorithm is extremely fast but generally produces poor-quality solutions.

Advantages

- Very simple to implement.
- Extremely fast.
- Serves as a baseline for comparison.

Limitations

- Ignores edge weights.
- Produces highly variable solutions.
- Lowest solution quality among the implemented algorithms.

2.2 Greedy Algorithm

The Greedy algorithm first selects the edge having the largest weight and places its endpoints into opposite partitions. Afterwards, vertices are inserted one at a time.

For every unassigned vertex, the total edge weight connecting it to each partition is computed. The vertex is then placed into the partition that immediately maximizes the contribution to the cut.

Since every decision is made using only local information, the algorithm is deterministic and significantly improves upon the Randomized approach.

Advantages

- Produces much better solutions than random assignment.
- Simple and computationally efficient.
- Deterministic.

Limitations

- Early decisions cannot be revised.
- May become trapped in poor local constructions.

2.3 Semi-Greedy Algorithm

The Semi-Greedy algorithm introduces controlled randomness into the Greedy construction.

Instead of always selecting the single best candidate, a Restricted Candidate List (RCL) is created. The threshold for constructing the RCL is

$$\mu = w_{min} + \alpha(w_{max} - w_{min}),$$

where

- w_{min} denotes the minimum greedy score,
- w_{max} denotes the maximum greedy score,
- $\alpha \in [0, 1]$ controls the amount of randomness.

A candidate whose greedy value is at least μ becomes eligible for selection. One candidate is then chosen uniformly at random from the RCL.

When $\alpha = 0$, the method behaves almost greedily, whereas larger values of α encourage exploration of different solutions.

Advantages

- Avoids deterministic behavior.
- Produces higher-quality initial solutions.
- Provides diversity for GRASP.

2.4 Local Search

The Local Search algorithm improves an already constructed solution.

For every vertex, the gain obtained by moving it to the opposite partition is computed. The vertex with the highest positive improvement is flipped, and the process repeats until no improving move remains.

This hill-climbing procedure gradually improves the cut value while remaining computationally inexpensive.

Advantages

- Consistently improves constructive solutions.
- Converges quickly in practice.
- Easy to implement.

Limitations

- Can converge to a local optimum.
- Quality depends on the initial solution.

2.5 GRASP

GRASP (Greedy Randomized Adaptive Search Procedure) is a multi-start metaheuristic that repeatedly combines Semi-Greedy construction with Local Search.

Each iteration consists of two phases:

1. Construct an initial solution using the Semi-Greedy algorithm.
2. Improve the solution using Local Search.

After several iterations, the best solution encountered is reported.

Because every iteration begins from a different randomized construction, GRASP explores multiple regions of the search space and usually produces significantly better solutions than any single constructive heuristic.

3. Experimental Setup

The algorithms were implemented in Python and evaluated using the benchmark graphs supplied with the assignment.

The following parameters were used throughout the experiments.

Parameter	Value
Programming Language	Python
Randomized Iterations	10
GRASP Iterations	10
α	0.3
Benchmark Graphs	54

4. Comparison of Algorithms

The experimental results demonstrate a clear progression in solution quality from simpler heuristics to more advanced metaheuristic approaches.

The Randomized algorithm consistently produced the weakest solutions because it ignored the graph structure during partition construction. Although computationally inexpensive, its average cut values were significantly lower than those obtained by the remaining algorithms.

The Greedy algorithm substantially improved the solution quality by making locally optimal decisions based on edge weights. However, since these decisions are irreversible, the algorithm occasionally became trapped in poor constructions.

Introducing controlled randomness through the Semi-Greedy algorithm generally resulted in better initial solutions than the purely Greedy approach. The Restricted Candidate List allowed exploration of multiple promising alternatives while still preserving greedy behavior.

Applying Local Search further improved almost every Semi-Greedy solution. By repeatedly performing improving vertex moves, Local Search effectively refined the partition until reaching a local optimum.

Among all implemented algorithms, GRASP consistently produced the highest cut values. Because it repeatedly generated different Semi-Greedy solutions and refined each using Local Search, it explored a larger portion of the search space and was able to obtain solutions that were much closer to the known best benchmark values.

Overall, the results indicate that combining randomized construction with iterative improvement provides significantly better performance than purely constructive heuristics.

5. Conclusion

Five heuristic algorithms for the Maximum Cut problem were implemented and evaluated on the benchmark graph set.

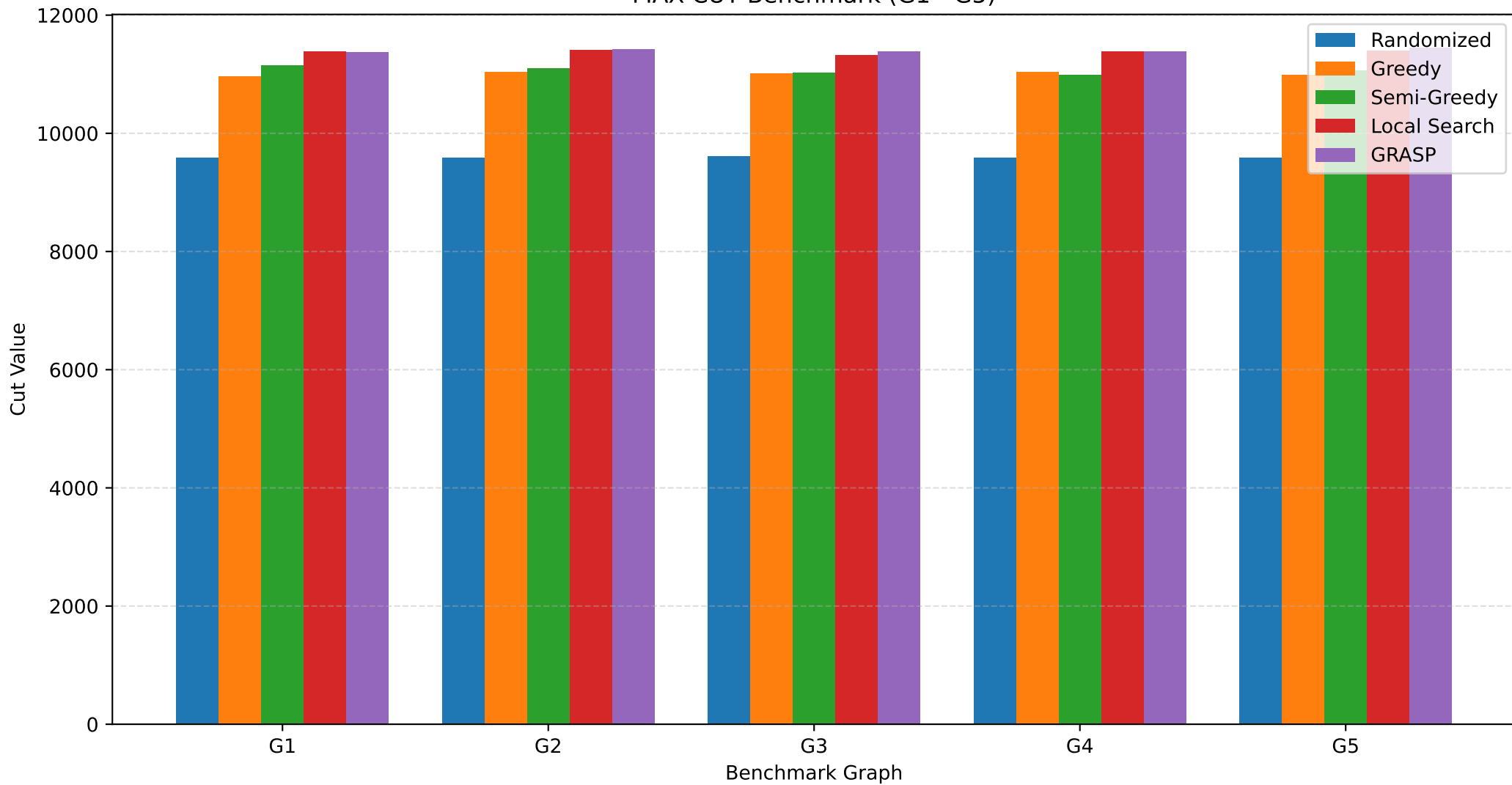
The Randomized algorithm served as a simple baseline, while the Greedy algorithm demonstrated that incorporating edge-weight information substantially improves solution quality. The Semi-Greedy algorithm further enhanced the constructive phase by introducing controlled randomness. Local Search consistently refined the constructed solutions, and GRASP achieved the best overall performance by combining Semi-Greedy construction with iterative local improvement.

The experimental results demonstrate the effectiveness of hybrid metaheuristic techniques for solving difficult combinatorial optimization problems such as MAX-CUT.

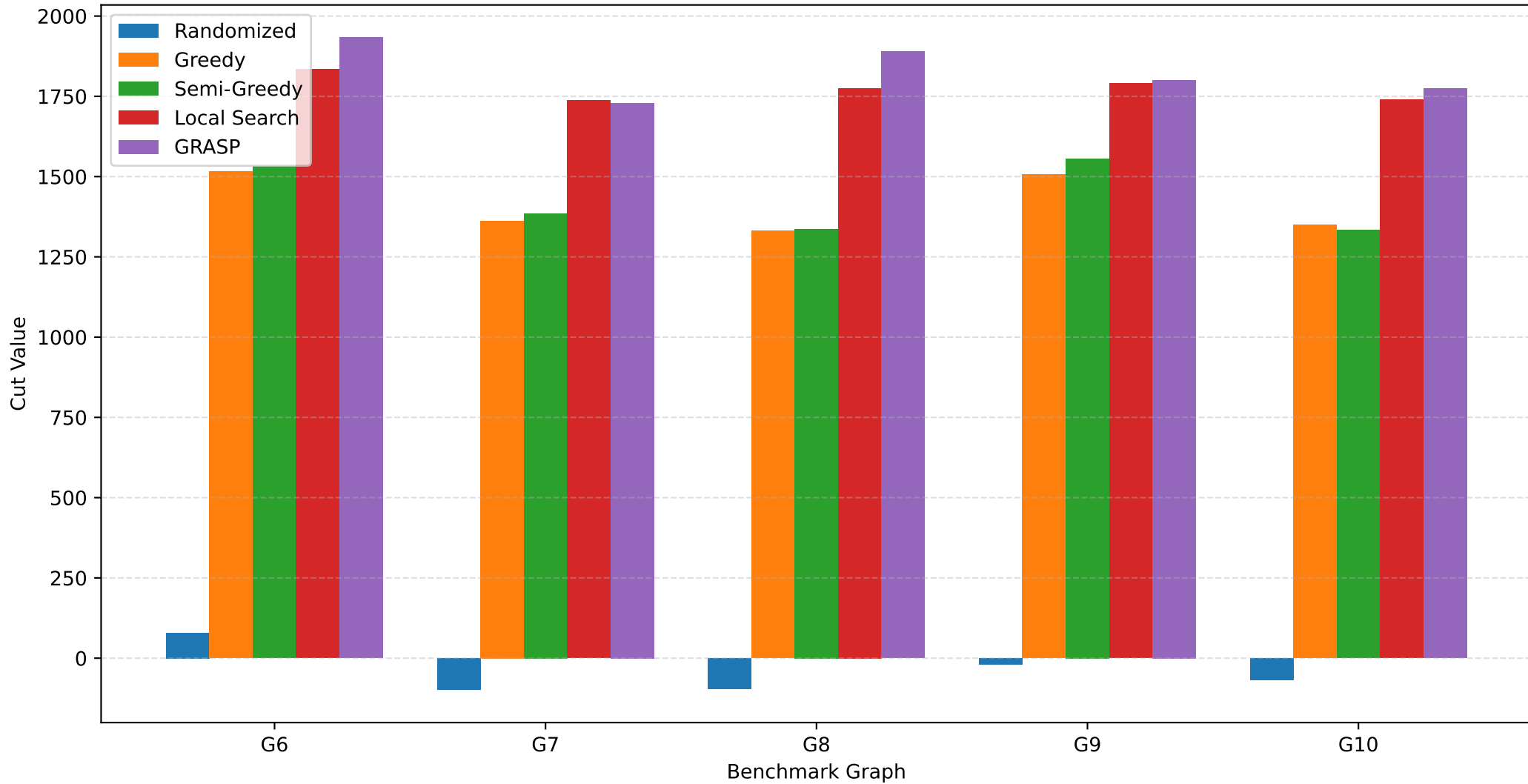
Plots

The comparison plots for all benchmark graphs are attached in the following pages.

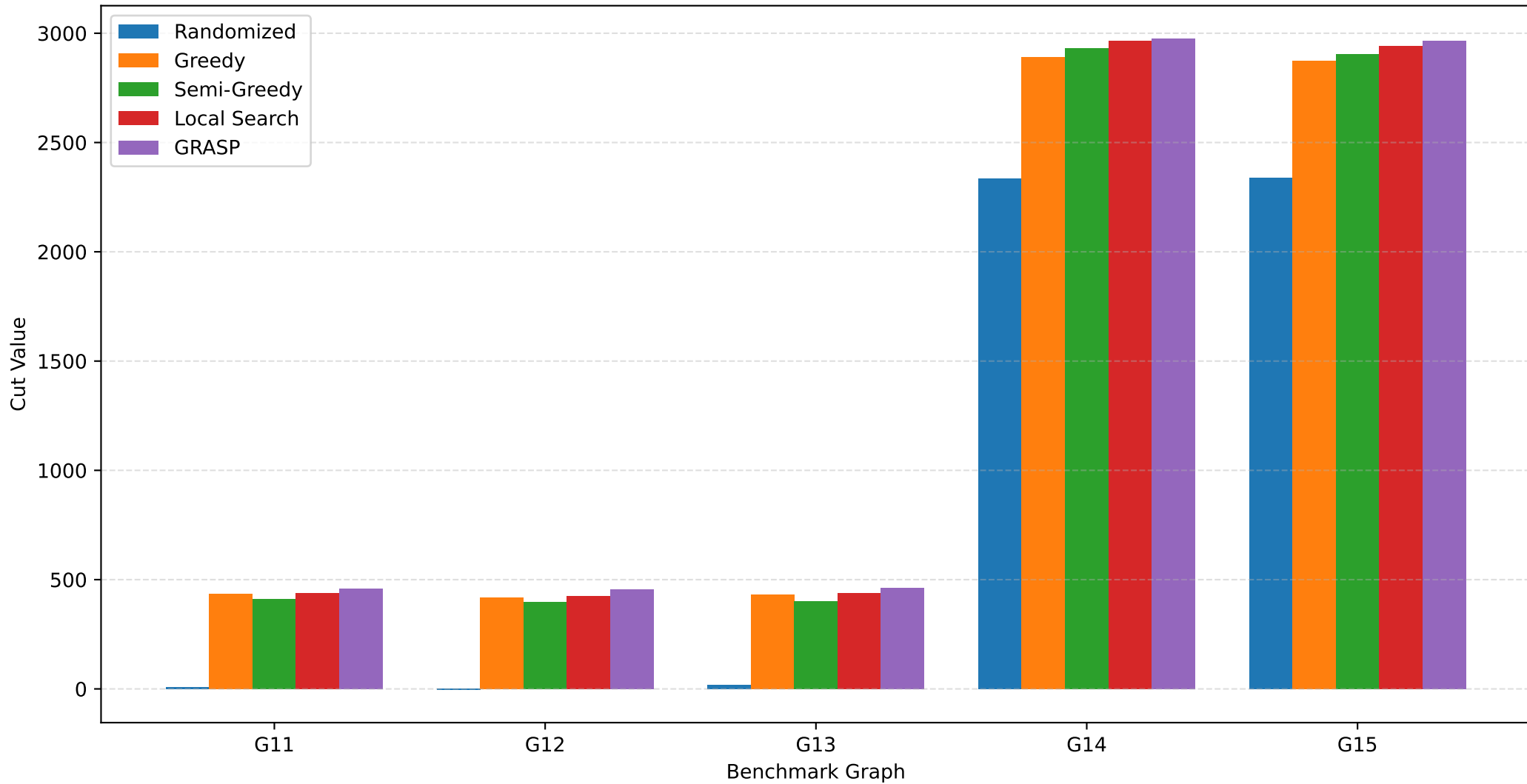
MAX-CUT Benchmark (G1 - G5)



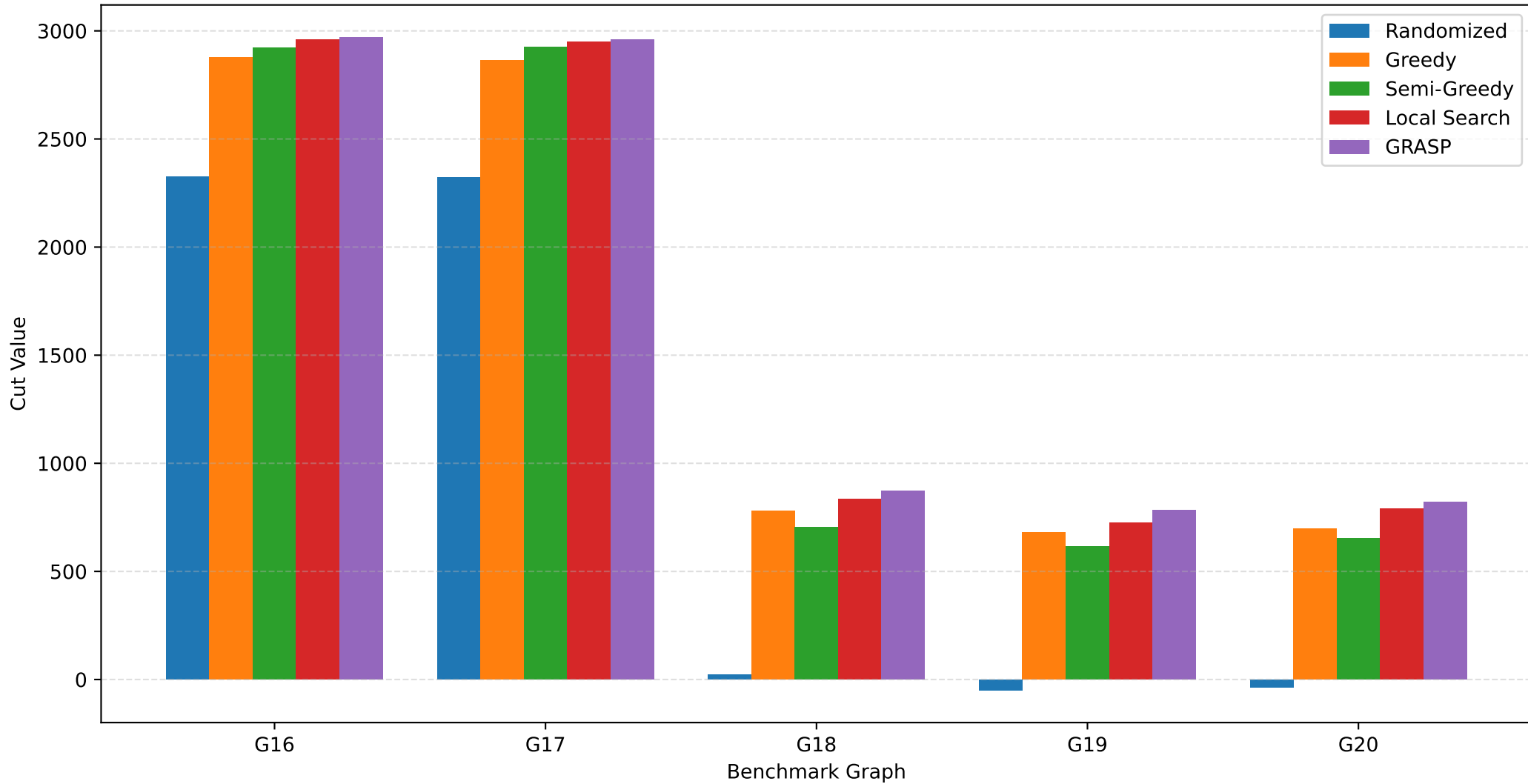
MAX-CUT Benchmark (G6 - G10)



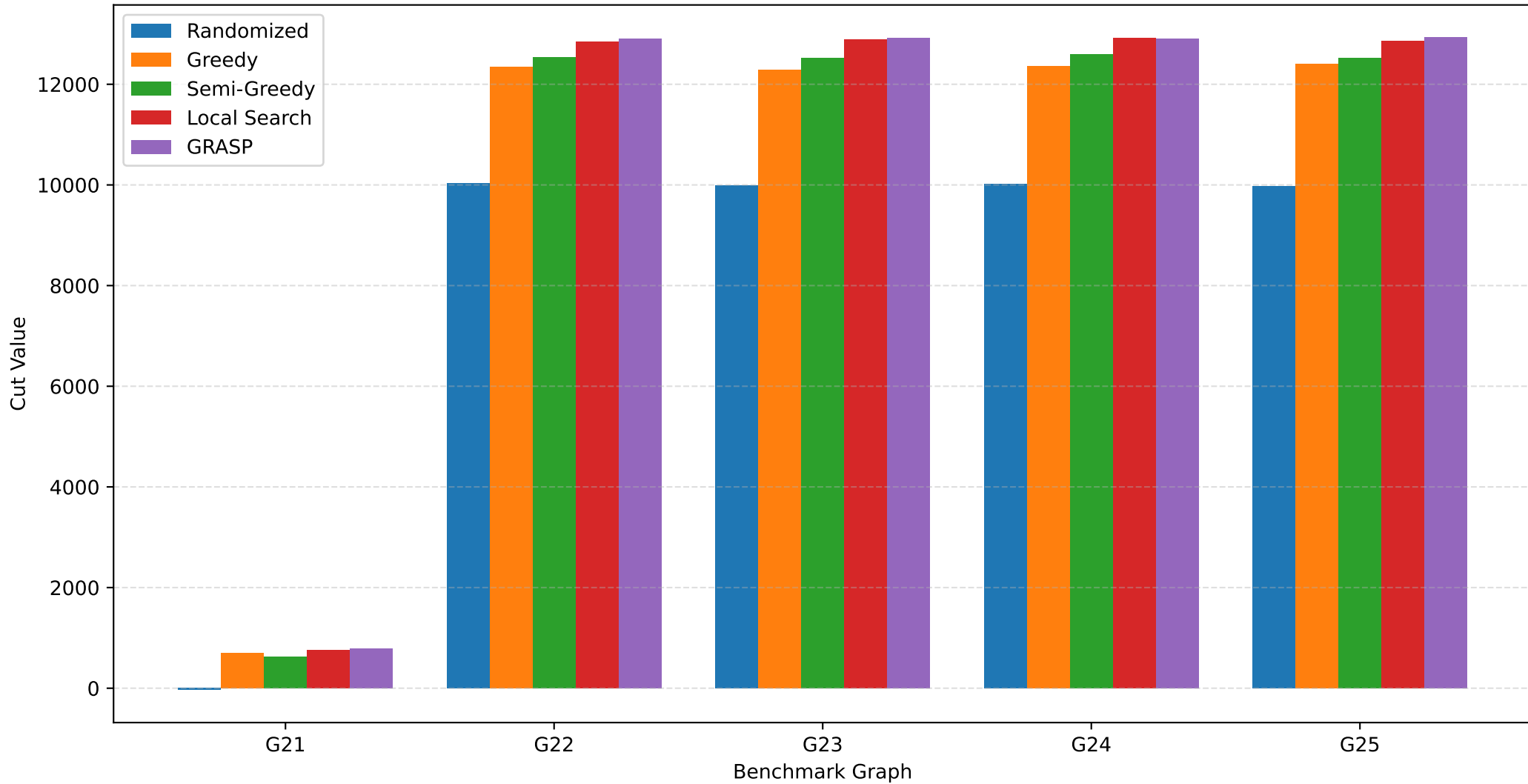
MAX-CUT Benchmark (G11 - G15)



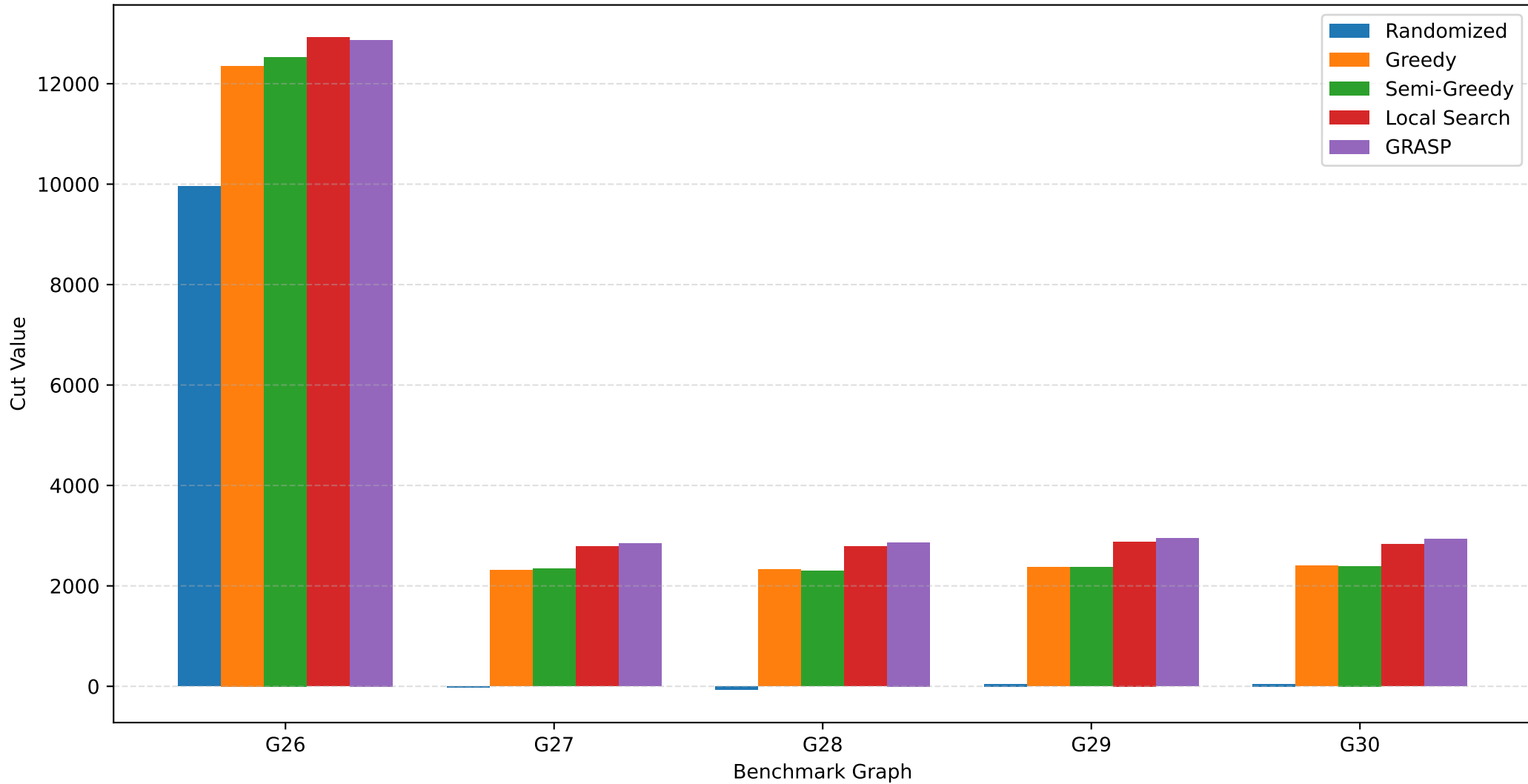
MAX-CUT Benchmark (G16 - G20)



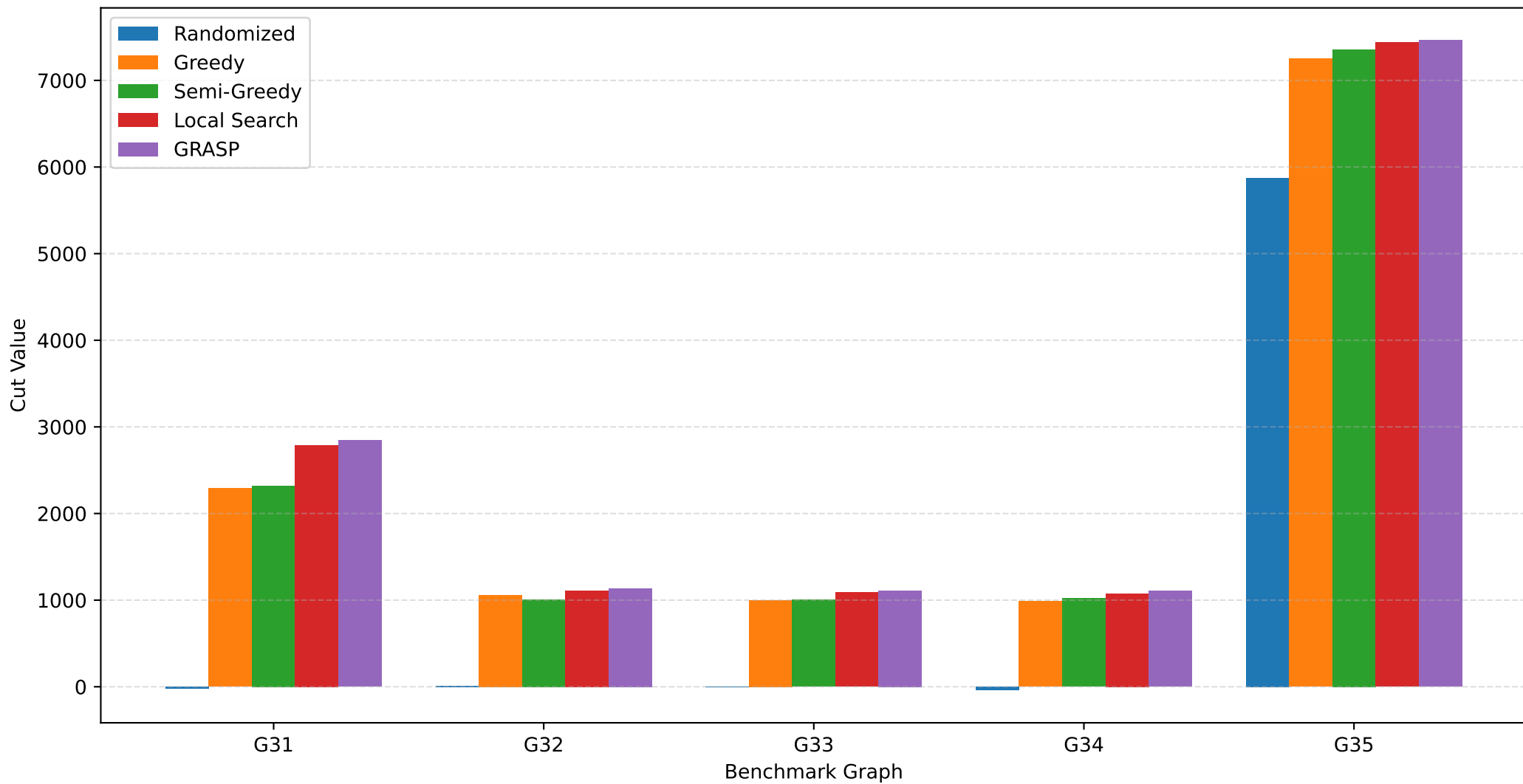
MAX-CUT Benchmark (G21 - G25)



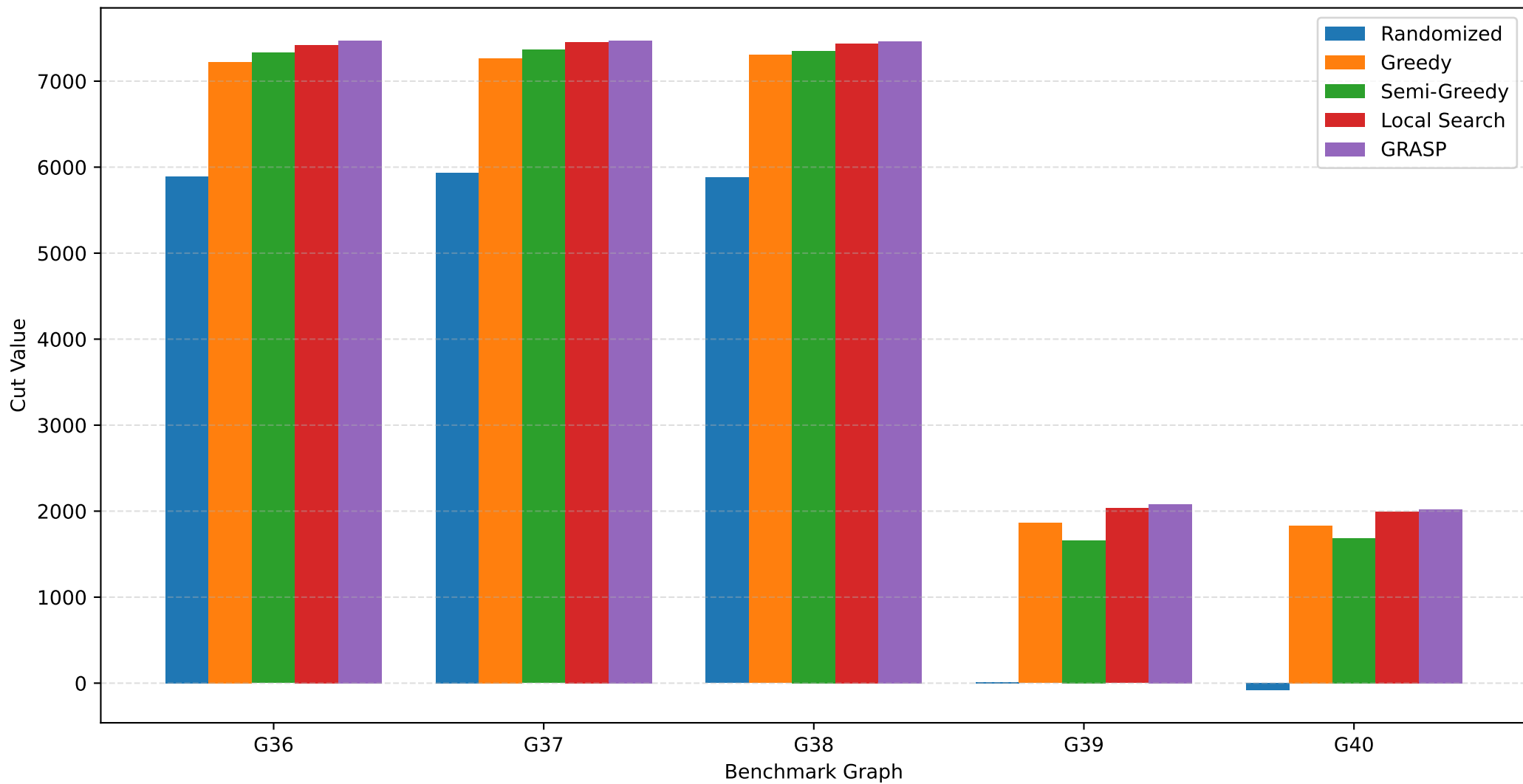
MAX-CUT Benchmark (G26 - G30)



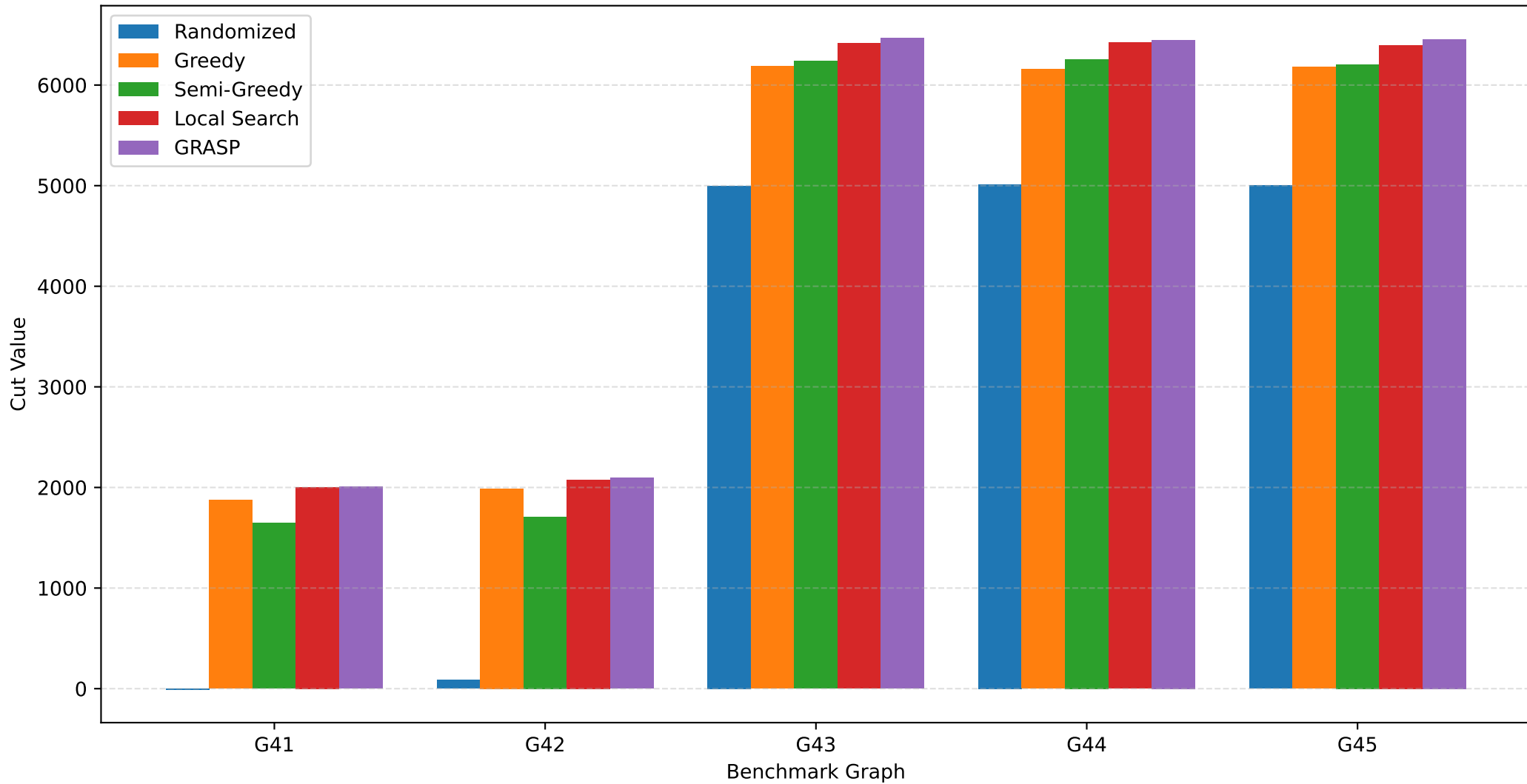
MAX-CUT Benchmark (G31 - G35)



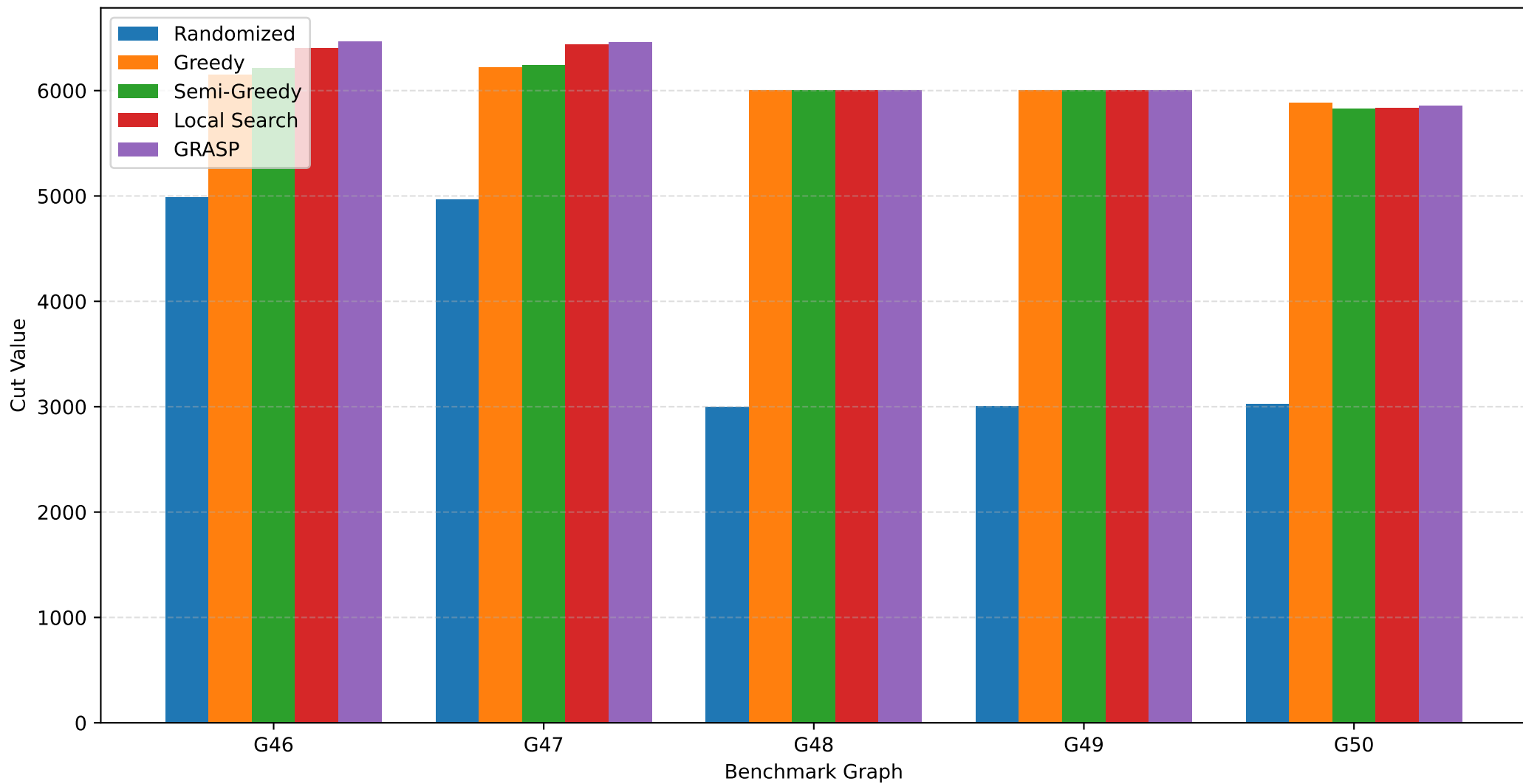
MAX-CUT Benchmark (G36 - G40)



MAX-CUT Benchmark (G41 - G45)



MAX-CUT Benchmark (G46 - G50)



MAX-CUT Benchmark (G51 - G54)

